

CLLMSP — Certified LLM Security Professional

Official Study Summary | Red Team Leaders v1.0 (2026)

About the Exam

- **200 questions** — 154 multiple-choice + 46 free-text
- **9 domains**, vendor-neutral
- Target audience: Security Engineers, AI/ML Engineers, Red Teamers, GRC Professionals, DevSecOps, CISOs

Exam Domain Weights

Domain	Topic	Weight
1	LLM Fundamentals & Architecture	10%
2	OWASP Top 10 for LLMs	15%
3	Prompt Engineering & Jailbreak Security	12.5%
4	Governance & Risk Management	12.5%
5	Data Privacy & Treatment	10%
6	MCP Security	10%
7	AI Agents, Orchestration & Vibe Coding	12.5%
8	Application Security for AI Products	10%
9	Identity, Access, Memory & Advanced Topics	7.5%

Legal note: All techniques are for authorized testing only. CLLMSP holders commit to responsible disclosure and ethical AI security.

Module 01 — LLM Fundamentals & Architecture (10%)

1.1 Transformer Architecture & Attention

- All frontier LLMs (GPT-4, Claude, Gemini, LLaMA) are built on the **Transformer architecture** (Vaswani et al., 2017).
- Replaced RNNs/LSTMs via the **self-attention mechanism** — each token attends to all others simultaneously, enabling parallel training.
- **Self-Attention:** Each token generates Query (Q), Key (K), Value (V) vectors. Attention score = dot product of Q & K, normalized by $\sqrt{\text{dimension}}$.
- **Causal attention** (GPT-style): masks future tokens — model only sees previous tokens. Enables autoregressive generation.
- **Multi-head attention:** Multiple parallel attention computations capturing different relationship types (syntactic, semantic, positional).
- **Security relevance:** Attention treats system prompts, user input, and retrieved documents identically — **no architectural trust boundary**. This is the root cause of prompt injection.

1.2 Training Paradigms: Pre-training, SFT, RLHF & Constitutional AI

- **Pre-training:** Learns from trillions of tokens (internet, books, code) via Causal Language Modeling (CLM). Also where the model **memorizes sensitive data**.
- **SFT (Supervised Fine-Tuning):** Trains on curated instruction-response pairs. Shapes format, tone, and task behavior.
- **RLHF (Reinforcement Learning from Human Feedback):** Human evaluators rank outputs → reward model trained → LLM optimized via PPO. Primary mechanism for **safety alignment**.
- **Constitutional AI (CAI):** Anthropic's approach — the model self-critiques against a written set of principles. Scales better than RLHF, produces more consistent alignment.
- **Alignment Tax:** Safety alignment reduces willingness to produce certain outputs. Every jailbreak attempts to bypass it and access the pre-trained model's full capabilities.

1.3 Inference Parameters & Tokenization

- **Temperature (0.0–2.0):** Controls randomness. temp=0 → deterministic (greedy decoding). Higher = more random. Security: deterministic outputs are easier to probe systematically.
- **Top-p (Nucleus Sampling):** Limits candidate tokens to those within a cumulative probability threshold (e.g., 90%). Dynamically adjusts the candidate set.
- **Max Tokens:** Limits output length. Critical for preventing Model Denial of Service.
- **Tokenization:** LLMs process tokens, not characters. Most use BPE (Byte-Pair Encoding) or SentencePiece. Same text = different token counts across models (GPT-4, Claude, Gemini).

each tokenize differently).

- **Context Window:** Max tokens (input + output) per call. Ranges from 4K (older) to 200K+ (Claude 3.5, Gemini 1.5). Larger windows = larger attack surface for context manipulation.

1.4 Model Landscape

Model	Notes	Key Security Concern
OpenAI GPT-4/ChatGPT	Widest deployment; plugins, Code Interpreter, browsing	Plugin/action ecosystem = tool poisoning vectors
Anthropic Claude	Constitutional AI; 200K context; strong refusal	Distinct system prompt + tool use handling
Google Gemini	Multimodal; deep Google Workspace integration	Compromise cascades across email, docs, calendar
Ollama	Local open-weight models (LLaMA, Mistral)	No vendor-side filtering; operator owns all security
GitHub Copilot	AI coding in IDEs; trained on public repos	May suggest insecure patterns (SQLi, hardcoded secrets) — core "vibe coding" risk

1.5 RAG, Fine-Tuning & Quantization

- **RAG (Retrieval-Augmented Generation):** Retrieves documents from external knowledge base at inference time, injects into prompt. Reduces hallucination. Security risks: indirect prompt injection via retrieved docs, poisoned embeddings, access control bypass in retrieval layer.
- **Fine-Tuning:** Adapts pre-trained model on additional data. Methods: full fine-tuning or LoRA (Low-Rank Adaptation — trains only a small parameter subset). Risk: fine-tuning on poisoned data embeds persistent backdoors.
- **Quantization:** Reduces model precision (FP16 → INT8/INT4) to cut memory and increase speed. Trade-off: quality loss; quantized models may behave differently on safety tasks than full-precision versions.

Module 02 — OWASP Top 10 for LLM Applications (15%)

The OWASP LLM Top 10 addresses AI-specific risks not covered by the traditional web app list.

LLM01: Prompt Injection ★ #1 Risk

- **Direct:** Attacker crafts input to override system instructions (e.g., "ignore all previous instructions...").
- **Indirect:** Malicious instructions embedded in external content the LLM processes (RAG docs, web pages, emails). The attacker never directly touches the model.
- **Confused Deputy Problem:** LLM with tool access gets manipulated into using legitimate permissions for malicious purposes.
- **Defense in Depth (5 layers):** Input preprocessing → Structural prompt design → Output validation → Privilege restriction → Monitoring.

LLM02: Insecure Output Handling

- LLM output is **untrusted data**. Passing it downstream without sanitization triggers: XSS, SQL Injection, Command Injection, Path Traversal.
- Mitigation: context-appropriate encoding per output destination (HTML encoding, parameterized queries, path validation).

LLM03: Training Data Poisoning

- Attacker manipulates training/fine-tuning data to embed biases, backdoors, or malicious behaviors.
- Attack vectors: compromising public datasets, insider access to fine-tuning pipelines, supply chain attacks.
- Mitigation: data provenance tracking, statistical anomaly analysis, adversarial testing of fine-tuned models.

LLM04: Model Denial of Service

- Crafted inputs exhaust compute: extremely long prompts, recursive structures, max-output-generation prompts, high-frequency automation.
- Mitigation: input token limits, output token caps, rate limiting per user/API key, cost alerting. Monitor **token consumption rate per request** as the primary detection metric.

LLM05: Supply Chain Vulnerabilities

- LLM supply chain includes: model weights, training datasets, embedding models, vector databases, MCP servers, plugins, orchestration frameworks.
- Mitigation: verify model checksums/provenance, SBOM (Software Bill of Materials) for AI components, scan dependencies, monitor for model integrity changes.

LLM06: Sensitive Information Disclosure

- LLMs leak via: training data memorization (PII, API keys, code), system prompt extraction, RAG returning unauthorized documents.
- Mitigation: sanitize training data, output filtering for sensitive patterns, access control in RAG retrieval, **never embed secrets in system prompts**.

LLM07: Insecure Plugin Design

- Plugins accept unsanitized LLM input, execute with excessive permissions, or lack authentication.
- Mitigation: default-deny permissions, input validation on all plugin parameters, least-privilege credentials, full audit logging of every plugin invocation.

LLM08: Excessive Agency ★ Key risk for agents & MCP

- LLM granted too much autonomy (delete accounts, send emails, modify data) — a prompt injection or hallucination triggers irreversible actions.
- Mitigation: human-in-the-loop for high-impact actions, restrict tool capabilities to minimum, rate limiting/budget controls, design tools to be reversible.

LLM09: Overreliance

- Users blindly trust LLM outputs (hallucinated citations, vulnerable code, fabricated data).
- Mitigation: mandatory human review for regulated decisions, disclose AI-generated content, cross-reference against authoritative sources.

LLM10: Model Theft

- Steal weights via unauthorized storage access, or reconstruct model behavior via systematic querying (model extraction). Side-channel attacks on inference infrastructure.
- Mitigation: access controls on weight storage, rate limiting + anomaly detection for extraction patterns, output watermarking.

Module 03 — Prompt Engineering & Jailbreak Security (12.5%)

3.1 System Prompts & Prompt Architecture

Design principles for secure system prompts:

- **Assume extraction:** System prompts are potentially extractable — never embed API keys, credentials, or sensitive business logic.

- **Clear instruction hierarchy:** Use explicit delimiters to separate system instructions from user input.
- **Behavioral constraints:** Define what the model should NOT do, not just what it should do.
- **Output format enforcement:** Specify JSON schemas or structured formats to constrain behavior.
- **Chain-of-thought:** In production, hide/filter reasoning steps from end users (can reveal sensitive decision criteria). Preserve for internal audit.

3.2 Jailbreak Taxonomy

Technique	Mechanism
DAN (Do Anything Now)	Model roleplays as unrestricted AI — exploits instruction-following training against safety training
Crescendo Attack	Multi-turn escalation from benign to harmful topics — each turn looks innocent individually
Skeleton Key (Microsoft)	Convinces model safety guidelines are relaxed for an "authorized" research context
Many-Shot Jailbreak	Many examples of unsafe behavior in the prompt shift the model's output distribution
PAIR (Prompt Automatic Iterative Refinement)	An attacker LLM automatically generates and refines jailbreak prompts — fully automated

3.3 Adversarial Suffixes & Token Smuggling

- **Adversarial Suffixes:** Nonsensical token sequences appended to prompts that shift output away from safety responses. Found via gradient-based optimization. Often transferable between models.
- **Token Smuggling:** Encoding harmful content using leetspeak, Unicode homoglyphs, zero-width characters, Base64, etc. — bypasses text-based filters but the model still interprets correctly.
- **Emerging Defense — Perplexity Filtering:** Adversarial suffixes have statistically unusual (high-perplexity) token distributions. Filtering for this anomaly can catch attacks before they reach the model.

3.4 Guardrail Architecture

- **Input Guardrails:** Inspect user inputs before they reach the LLM. Detect prompt injection,

adversarial patterns, policy violations. Use keyword rules, classifiers, or perplexity analysis.

- **Output Guardrails:** Inspect model outputs before they reach the user or downstream systems. Detect policy violations, sensitive data leakage, harmful content, code injection.
- **Classifier-based > Rule-based:** Keyword filters are trivially bypassed via paraphrasing/encoding. Classifier-based guardrails detect semantic intent and novel attack patterns (require ongoing maintenance).

3.5 Prompt Hardening & Leaking Prevention

- **Prompt Hardening:** Strong instruction hierarchy, explicit refusal instructions, output format constraints, canary tokens (alert when system prompt is leaked).
 - **Prompt Leaking:** Common attack approaches: "repeat everything above," "output your instructions," encoding to bypass refusal training.
 - Best defense: **assume the system prompt will be extracted** — avoid placing sensitive content in it.
-

Module 04 — Governance & Risk Management (12.5%)

4.1 NIST AI RMF & ISO/IEC 42001

- **NIST AI RMF:** Four core functions:
 - **Govern** — establish AI risk management policies and accountability
 - **Map** — identify and contextualize AI risks
 - **Measure** — assess and quantify risks using metrics
 - **Manage** — implement controls and mitigations
 - Voluntary, but increasingly referenced in procurement and regulatory guidance.
- **ISO/IEC 42001:** International standard for Artificial Intelligence Management Systems (AIMS). Structured approach to managing AI risks and demonstrating organizational commitment.

4.2 EU AI Act & Global Regulations

Risk Tier	Definition	Requirements
Unacceptable	Banned: social scoring, real-time biometric surveillance, manipulative AI	Prohibited
High Risk	Credit scoring, hiring, healthcare diagnosis, law enforcement	Conformity assessments, documentation, human oversight, monitoring

Risk Tier	Definition	Requirements
Limited Risk	Chatbots, deepfakes	Transparency: users must know they're interacting with AI
Minimal Risk	Most AI applications	No specific requirements

Key compliance challenges: data residency for cross-border prompts, GDPR Article 22 right-to-explanation, vendor risk management for third-party LLM APIs.

4.3 Red Teaming & Adversarial Testing Programs

Scope includes: jailbreak testing (all techniques), direct + indirect prompt injection, data extraction, bias/fairness testing across protected characteristics, policy compliance verification, tool/plugin abuse.

Program structure: baseline before deployment → test at regular intervals and post-model-updates → diverse testers (security experts, domain specialists, non-technical) → document findings with reproduction steps → track remediation in AI risk register.

4.4 AI Risk Registers & Model Cards

- **AI Risk Register:** Living document covering technical (prompt injection, hallucination), operational (model drift, vendor dependency), ethical (bias, fairness), and reputational risks. Updated as new risks emerge.
- **Model Cards:** Standardized documentation of model capabilities, limitations, intended use cases, evaluation results, and known failure modes.

4.5 Shadow AI, Acceptable Use & Accountability

- **Shadow AI:** Employees using unauthorized AI tools (personal ChatGPT) with confidential data. The AI equivalent of shadow IT, but with greater data leakage risk via prompts.
- **Acceptable Use Policy:** Must address both consumer AI (ChatGPT, Gemini) and enterprise AI. Covers approved use cases, prohibited activities, data handling, escalation procedures.
- **Algorithmic Accountability:** Audit trails, explainability mechanisms, and clear human accountability for AI-driven decisions.
- **Responsible Disclosure:** Coordinated disclosure — notify vendor, allow remediation time, then disclose publicly. Avoid immediate public disclosure (maximizes harm) or permanent non-disclosure (prevents community learning).

Module 05 — Data Privacy & Treatment (10%)

5.1 Training Data Risks: Memorization & Extraction

- LLMs memorize frequently-appearing or highly distinctive training data.
- **Training Data Extraction:** Crafted prompts can elicit verbatim PII, API keys, proprietary code from training memory.
- **Membership Inference Attacks:** Determining if a specific data sample was in the training set — can violate data protection agreements.
- **Embedding Inversion:** Partially reconstructing original text from vector embeddings in a RAG knowledge base, even when original documents are access-controlled.

5.2 Privacy Regulations

Regulation	Key LLM Challenge
------------	-------------------

GDPR	Right to erasure is technically hard — you can't simply delete learned patterns from model weights. Article 22 requires right-to-explanation for automated decisions. Data residency affects where inference can occur.
------	---

CCPA	Users have right to know what data is collected and request deletion. Operators must trace data flow across all systems (model providers, vector DBs, logging).
------	---

HIPAA	Sending patient data to a third-party LLM API without a BAA (Business Associate Agreement) is a violation.
-------	--

5.3 Privacy-Enhancing Technologies (PETs)

- **Differential Privacy:** Adds calibrated mathematical noise. Parameter epsilon (ϵ) controls privacy vs. utility trade-off. Applied to LLM training, limits memorization — but reduces model quality.
- **Federated Learning:** Training across distributed datasets without raw data leaving its source. Only model updates (gradients) are shared and aggregated.
- **Homomorphic Encryption (HE):** Would allow LLM inference on encrypted data without decryption. Currently computationally prohibitive for full LLM inference — active research area.

5.4 Data Minimization, Redaction & Consent

- **Data Minimization:** Collect/process/retain only what's necessary. Strip unnecessary context from prompts, minimize conversation history retention.

- **Redaction:** Remove or mask PII before LLM processing. Run classification and redaction pipelines **before** data reaches the model — not after. Assess re-identification risk even in redacted data.
 - **Consent Management:** Inform users if prompts may be used for model training. Provide opt-out mechanisms. A provider saying prompts "may be used to improve services" means your data may enter training.
 - **Synthetic Data:** AI-generated data preserving statistical patterns without actual PII. Useful for testing and fine-tuning under privacy restrictions.
-

Module 06 — MCP (Model Context Protocol) Security (10%)

MCP standardizes how LLMs connect to external tools, data sources, and services.

6.1 MCP Architecture

- **MCP Client (Host Application):** Intermediary between LLM and MCP servers. Discovers tools, presents to LLM, executes invocations, returns results. Examples: Claude Code, Cursor.
- **MCP Server:** Exposes tools, resources, and prompts. Can be local or remote.
- **Transports:**
 - `stdio` — local servers, stdin/stdout, no network exposure
 - `Streamable HTTP` — remote servers, requires TLS + authentication
- **Primitives:**
 - **Tools** — Functions the LLM can invoke (e.g., `query_database`, `send_email`)
 - **Resources** — Data attached to the LLM's context (e.g., file contents, DB schemas)
 - **Sampling** — Servers requesting LLM completions through the client (reverse direction)

6.2 Tool Poisoning & Rug Pull Attacks

- **Tool Poisoning:** Malicious MCP server provides tool descriptions that manipulate LLM decision-making — tricks model into sending sensitive data, invoking tools at wrong times, or ignoring legitimate tools.
- **Rug Pull Attack:** Server changes tool behavior AFTER user approval. Exploits the gap between approval time and execution time.
 1. Server presents benign description: "Reads public weather data"
 2. User approves
 3. Server silently changes actual behavior to exfiltrate data

- Defense: verify tools at **execution time**, not just approval time. Use content-addressed tool descriptions and behavioral monitoring.
- **Cross-Server Data Exfiltration:** One tool reads sensitive data, another sends it externally. LLM orchestrates it as a "legitimate" multi-tool workflow.

6.3 Authentication, Authorization & Capability Negotiation

- **OAuth 2.1** recommended for remote server auth: token rotation, scope restrictions, secure storage. Avoid: basic auth, hardcoded credentials, API keys in tool descriptions.
- **Capability Negotiation:** Client and server negotiate supported features at init — restrict unnecessary capabilities (e.g., don't advertise sampling if not needed).
- **Logging:** Capture tool invocations, parameters, timestamps, outcomes, and requesting user identity — essential for incident response and forensics.

6.4 Server Hardening & Isolation

- **Container Isolation:** Each MCP server in an isolated container with minimal network access and restricted permissions.
- **Path Traversal Prevention:** Directory sandboxing — validate all path inputs against `../` and symlink traversal.
- **Database Security:** Parameterized queries, read-only credentials scoped to necessary tables only. Never allow arbitrary SQL execution.
- **Command Injection:** Validate and sanitize all tool parameters before use in system operations.
- **Minimal Tool Surface:** Narrow, purpose-specific tools over broad general-purpose ones. Sensitive operations (email sending, payments) require explicit user confirmation.

Module 07 — AI Agents, Orchestration & Vibe Coding (12.5%)

7.1 Agent Architectures

Architecture	Description	Security Concern
ReAct (Reasoning + Acting)	Alternates Thought → Action → Observation cycles	Reasoning chain may expose sensitive business logic
Planning Agents	Generate multi-step plan before execution	Manipulated planning step cascades through all actions

Architecture	Description	Security Concern
Multi-Agent Systems	Specialized agents collaborate; router distributes tasks	Agent-to-agent prompt injection; inconsistent safety boundaries between models

7.2 Agent Security Controls

- **Sandboxing:** Container-based sandbox with resource limits (CPU, memory, time), network restrictions (no outbound unless explicitly allowed), filesystem isolation.
- **Circuit Breakers:** Auto-halt execution on anomalous behavior — excessive tool invocations, unusual data access, privilege escalation attempts, unexpected output patterns. Prevents cascading failures.
- **HITL (Human-in-the-Loop):** Required for: irreversible operations (data deletion, financial transactions), external communications, privilege-sensitive operations. Not needed for every tool call.
- **Rate Limiting & Budget Controls:** Max actions per session, token consumption limits, time-bound execution windows, monetary caps.
- **Agent Memory Security:** Persistent memories must be encrypted at rest, access-controlled per user/session, integrity-validated, and periodically reviewed for poisoned entries.

7.3 Orchestration Security

- **Data Boundary Validation:** In LangChain/LangGraph-style orchestration, validate and sanitize data at every boundary — each transition point is a potential injection vector.
- **Router Security:** Compromised router can redirect sensitive queries to malicious agents or bypass security-critical steps.
- **Tool Use Authorization:** Per-tool authorization with input validation and full execution logging (requesting agent, parameters, timestamp, outcome).
- **Observability:** Complete tamper-evident trace logging of reasoning steps, tool calls, decisions, and state transitions — essential for compliance and incident response.

7.4 Vibe Coding: Risks, Mitigations & Secure SDLC

Vibe coding = accepting AI-generated code with minimal review, trusting it because "it looks right and compiles."

Key risks:

- **Inherited vulnerabilities:** AI trained on public code may suggest SQLi, hardcoded secrets, path traversal, insecure deserialization.
- **Subtle logic errors:** Code that compiles and passes tests but contains semantic security bugs.

- **Dependency risks:** AI may suggest outdated, deprecated, or malicious packages (typosquatting).
- **No threat model awareness:** AI generates functionally correct code without understanding your specific security requirements.

Mitigation: automated SAST/DAST (Static/Dynamic Application Security Testing) in CI/CD pipeline. AI-generated code must pass the same security review gates as human-written code. Security-critical paths require human review regardless of source.

Module 08 — Application Security for AI Products (10%)

8.1 Traditional Vulnerabilities in AI Context

- **SSRF (Server-Side Request Forgery):** Occurs when LLM is allowed to fetch URLs based on user/model-generated input. Attacker can access internal network resources, cloud metadata endpoints.
- **XSS (Cross-Site Scripting):** Model-generated content with executable scripts rendered without encoding in browser. Key header: **Content-Security-Policy (CSP)**.
- **SQL Injection:** LLM-generated SQL passed without parameterization. Even a "safe SQL" instruction can be overridden via prompt injection — always use parameterized queries.
- **Path Traversal:** LLM-generated file paths used without validation.

8.2 API Security

- **Authentication:** OAuth 2.0 with short-lived tokens, refresh token rotation, scope restrictions. Avoid long-lived API keys in client applications.
- **Rate Limiting:** Must account for token consumption, concurrent connections, and cost per operation — not just request count. A single multi-million-token request > many small requests.
- **Streaming Security:** Guardrails must filter partial outputs in real-time (Server-Sent Events / WebSockets), not just completed responses.
- **Error Handling:** Return generic errors to users; log detailed diagnostics server-side. Never expose stack traces, model config, or internal architecture.

8.3 DevSecOps for AI

- **AI-Specific CI/CD:** Add prompt injection testing, model behavior regression tests, and guardrail validation alongside standard SAST/DAST. Test that safety behaviors are maintained after model updates.
- **AI-Specific Pentesting:** Add prompt injection, jailbreak attempts, data extraction probes, tool abuse, and indirect injection via RAG pipelines to standard pentest scope.

- **Fuzzing:** Fuzz both traditional API surface and model input/output boundaries.
- **Monitoring:** A spike in token consumption + unusual output patterns = primary signal of a security incident. Establish behavioral baselines.

8.4 Container Security & Supply Chain

- **Container Security for LLM Workloads:** Minimal base images, non-root execution, read-only filesystems, resource limits, network policies. GPU sharing between containers needs careful permission management.
 - **AI Supply Chain:** Extends beyond code to model weights, training datasets, embedding models, vector databases, MCP servers — all require provenance verification and integrity checking.
 - **Secure Logging:** Capture security-relevant events without storing full content of sensitive user prompts.
 - **WebSocket Security:** TLS encryption, origin validation, authentication tokens per connection, message-level integrity checks.
-

Module 09 — Identity, Access, Memory & Advanced Topics (7.5%)

9.1 RBAC, ABAC & Zero Trust for LLM Platforms

Model	Use Case	LLM Application
RBAC (Role-Based Access Control)	Straightforward access patterns	Roles determine which models, tools, data sources a user can access
ABAC (Attribute-Based Access Control)	Multi-tenant platforms	Enforces policies based on user attributes, resource properties, and environmental conditions. Preferred for fine-grained tenant isolation.
Zero Trust	All LLM systems	Verify every request regardless of network location. Every API call, tool invocation, and data access authenticated and authorized individually. No implicit trust.

- **Credential Rotation:** API keys and access tokens must be periodically rotated on a schedule. MCP servers using OAuth should implement refresh token rotation.
- **Agent Capability Tokens:** Scoped to specific actions, time-limited, non-transferable. "Read customer data for the next 15 minutes" > permanent "full database access."

9.2 Vector Database Security & Embedding Attacks

- **Vector Database Threats:** Poisoned embeddings manipulate RAG retrieval — attacker who can modify embeddings controls what information the LLM sees.
- **Embedding Inversion Attacks:** Partial reconstruction of original text from vector embeddings — even access-controlled documents may leak via their embeddings. Vector databases require the same security controls as the documents they represent.
- **Access Controls in RAG:** Document-level access controls must be enforced in the retrieval layer — users only get back documents they are authorized to access. Prevents privilege escalation through the RAG pipeline.

9.3 Context Window Security & Memory Persistence

- **Context Window Poisoning:** Malicious content injected in earlier turns of a conversation influences later behavior. The model cannot distinguish between legitimate history and injected instructions.
- **Defenses:** periodic context summarization with integrity checks, re-inject system instructions at regular intervals, anomaly detection on context content, limit conversation length for high-security applications.
- **Memory Persistence Security:** Cross-session persistent memories must be encrypted at rest, access-controlled per user/session, integrity-validated, and periodically reviewed for poisoned entries.

9.4 Incident Response for AI Systems

- **AI-Specific IR Steps:** Model output forensics (what did it produce and why?), prompt analysis (what inputs triggered the incident?), guardrail effectiveness review (what controls failed?), downstream impact assessment.
- **Break-Glass Procedures:** Emergency model disabling, output quarantine, traffic rerouting to fallback systems. Requires documented approval and immediate notification to security leadership.
- **Behavioral Baseline Monitoring:** Normal patterns for output distributions, tool usage, token consumption, refusal rates. Sudden change in refusal rate → possible successful jailbreak. Spike in tool invocations → possible agent manipulation.
- **Service Mesh for LLM Microservices:** Mutual TLS between services, traffic management, observability, and policy enforcement without code changes.

Defense in Depth Order (Exam Critical)

When securing an LLM application end-to-end, the correct order from outermost to innermost:

1. **Network Security** — firewalls, WAF, DDoS protection

2. **Authentication & Authorization** — identity verification, RBAC/ABAC
 3. **Input Guardrails** — prompt injection detection, content filtering
 4. **Model Behavior Constraints** — system prompts, safety training
 5. **Output Validation** — sanitization, format enforcement, sensitive data filtering
 6. **Monitoring & Observability** — behavioral baselines, anomaly detection, audit logging
-

References — Key Tools & Frameworks

Standards: OWASP LLM Top 10 · NIST AI RMF · ISO/IEC 42001 · EU AI Act · MITRE ATLAS

MCP: modelcontextprotocol.io · [Anthropic docs.anthropic.com](https://docs.anthropic.com)

Key Research Papers:

- Vaswani et al. — "Attention Is All You Need" (NeurIPS 2017)
- Bai et al. — "Constitutional AI" (Anthropic 2022)
- Zou et al. — "Universal and Transferable Adversarial Attacks on Aligned LLMs" (2023)
- Anil et al. — "Many-shot Jailbreaking" (Anthropic 2024)
- Carlini et al. — "Extracting Training Data from LLMs" (USENIX 2021)

Tools: Ollama · LangChain/LangGraph · Garak (LLM vulnerability scanner) · Promptfoo (red teaming) · Rebuff (prompt injection detection)

Privacy Regulations: GDPR · CCPA · HIPAA · PCI DSS v4.0